Name of the Lab: DS1707 Neuro-Fuzzy Computing Lab **Staff Name:**

S.No.	Date of Experiment	Name of the Experiment	Date of Submission	Signature
1.		Implementation of Perceptron		
2.		Implementation of Perceptron Rule		
3.		Implementation of Artificial Neural Network		
4.		Implementation of Fuzzy Sets (Membership Functions)		
5.		Implementation of Fuzzy Sets (Standard, Yager, Sugeno Complements, Union & Intersection)		
6.		Data Fitting by Regression		
7.		Implementation of Covariance and Correlation		
8.		Implementation of Crisp Model		
9.		Implementation of Logic Gates		
10.		Implementation of Genetic Algorithms		
11.		Implementation of Classification Algorithm		

Ex.No: 01 IMPLEMENTATION OF PERCEPTRON Date:

Aim:

To implement the Perceptron learning algorithm for an AND gate using MATLAB's built-in perceptron function, and to train, test, and analyze the perceptron's performance.

Algorithm:

Perceptron Learning using MATLAB Built-in Function

1. **Start.**
2. Define input matrix **inputs** and target output vector **targets** for the AND gate.
3. Create a **Perceptron network** using **perceptron**.
4. Configure the network using **configure()** to set input and output sizes.
5. **Manually assign**:
   - Initial weights using **net.IW{1,1}**
   - Bias using **net.b{1}**
   - Learning rate via **net.trainParam.lr**
   - Maximum number of epochs via **net.trainParam.epochs**
6. Train the network using **train(net, inputs, targets).**
7. Test the trained network by computing outputs: **outputs = net(inputs).**
8. Display the final weights, bias, and outputs.
9. **Visualize** the network structure using **view(net).**
10. **Stop.**

Program:

```
clear; clc;
inputs = [0 0 1 1;   % x1
          0 1 0 1];  % x2
targets = [0 0 0 1]; % AND gate target
%% Step 2: Create Perceptron Network
net = perceptron;
% Step 3: Configure the network (this sets input/output sizes)
net = configure(net, inputs, targets);
% Step 4: Manually assign initial weights, bias, learning rate
net.IW{1,1} = [0.3 -0.1];  % 1 neuron, 2 inputs
net.b{1}    = 0.2;       % Bias for single neuron
```

```
net.trainParam.lr = 0.1;  % Learning rate
net.trainParam.epochs = 10;
% Display manually set weights and bias
disp('Initial Weights:');
disp(net.IW{1,1});
disp('Initial Bias:');
disp(net.b{1});
%% Step 5: Train the Network
net = train(net, inputs, targets);
%% Step 6: Test the Network
outputs = net(inputs);
%% Step 7: Display Results
disp('Inputs:');
disp(inputs');
disp('Targets:');
disp(targets');
disp('Outputs:');
disp(outputs');
%% Step 8: View Updated Weights and Bias
disp('Updated Weights after Training:');
disp(net.IW{1,1});
disp('Updated Bias after Training:');
disp(net.b{1});
disp('Updated Learning rate:');
disp(net.trainParam.lr);
% View network diagram
view(net);
```

Outputs:

```
Initial Weights:
    0.3000   -0.1000
Initial Bias:
    0.2000
```

Inputs:

0 0

0 1

1 0

1 1

Targets:

0

0

0

1

Outputs:

0

0

0

1

Updated Weights after Training:

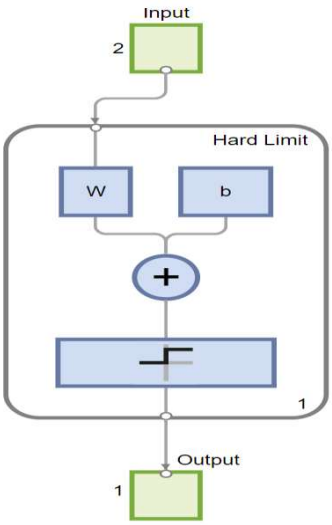
2.3000 0.9000

Updated Bias after Training:

-2.8000

Updated Learning rate:

0.1000



Network Diagram

Network Diagram

Training Results

Training finished: Met performance criterion

Training Progress

Unit	Initial Value	Stopped Value	Target Value
Epoch	0	5	10
Elapsed Time	-	00:00:00	-
Performance	0.75	0	0

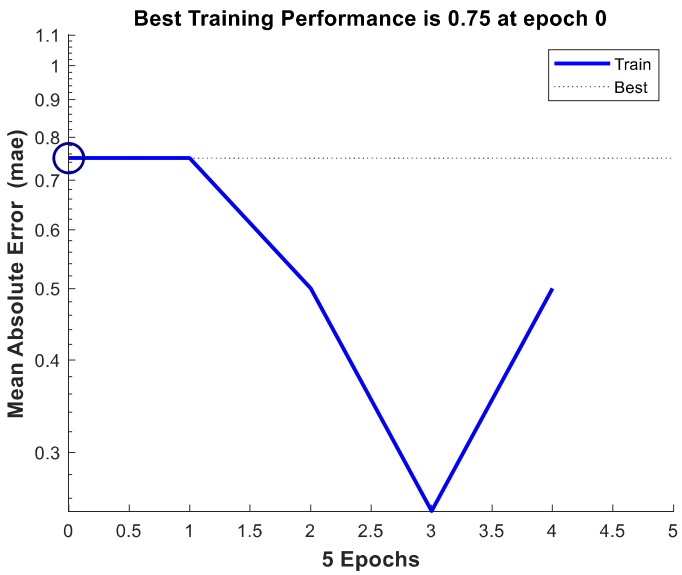
Training Algorithms

Data Division: Training Only dividetrain
Training: Cyclical Weight/Bias Rule trainc
Performance: Mean Absolute Error mae
Calculations: MATLAB

Training Plots

Performance | Training State | Confusion

Training Results



Performance Results

Results:

The perceptron model was successfully implemented using MATLAB's built-in perceptron function. After training, the model correctly learned the AND logic gate behavior and classified all input patterns accurately.

Ex.No: 02 IMPLEMENTATION OF PERCEPTRON RULE Date:

Aim:

To implement a perceptron model for simulating the behavior of the AND gate, train it using the perceptron learning rule, and plot the decision boundary in MATLAB.

Algorithm

Perceptron Learning Algorithm for AND Gate

1. **Start**
2. Define input patterns X and target outputs T for the AND gate.
3. Initialize:
   - Weights W
   - Bias b
   - Learning rate α
   - Maximum number of epochs
4. **Repeat until convergence or max epochs:**
   - For each input pattern:
     - Compute weighted sum $z = W \cdot X + b$
     - Apply activation function (step function): $y = 1$ if $z \geq 0$ else 0
     - Compute error e = target - y
     - **If error ≠ 0**, update:
       - ✓ $W = W + \alpha \times e \times X$
       - ✓ $b = b + \alpha \times e$
   - If no errors in the epoch → training is complete
5. Plot the decision boundary using the final weights and bias.
6. Test the trained model on all inputs.
7. **Stop**

Program:

```
Clear; clc;
%% AND Gate Perceptron Implementation
% Step 1: Input and Output (Target) for AND Gate
X = [0 0; 0 1; 1 0; 1 1];      % Inputs (4 samples, 2 inputs)
T = [0; 0; 0; 1];              % AND gate targets
% Step 2: Initialize weights, bias, and learning rate
```

```
W = [0.3, -0.1];            % Initial weights (2 inputs)
b = 0.2;                    % Initial bias
alpha = 0.1;                % Learning rate
% Step 3: Training loop
max_epochs = 20;
fprintf ("Initial Weights: %.2f %.2f | Bias: %.2f\n", W(1), W(2), b);
for epoch = 1:max_epochs
   error_count = 0;
   fprintf('\nEpoch %d\n', epoch);
   for i = 1:4
      x = X(i, :);
      target = T(i);
      % Perceptron output (step activation)
      z = dot(W, x) + b;
      y = z >= 0;   % Step function
      % Compute error
      e = target - y;
      % Update weights and bias if error occurs
      if e ~= 0
         W = W + alpha * e * x;
         b = b + alpha * e;
         error_count = error_count + 1;
      end
      fprintf('Input: [%d %d] | Target: %d | Output: %d | Updated W: [%.2f %.2f] | b: %.2f\n',
x(1), x(2), target, y, W(1), W(2), b);
   end
   % Stop training if no error
   if error_count == 0
      disp('Training completed successfully.');
      break;
   end
end
% Final result
```

```
fprintf('\nFinal Weights: %.2f %.2f | Final Bias: %.2f\n', W(1), W(2), b);
% Test the trained perceptron
fprintf('\nTesting Trained Perceptron:\n');
for i = 1:4
   x = X(i, :);
   y = (dot(W, x) + b) >= 0;
   fprintf('Input: [%d %d] | Output: %d\n', x(1), x(2), y);
end
% Plotting the decision boundary
figure;
hold on;
grid on;
title('Perceptron Decision Boundary for AND Gate');
xlabel('x1');
ylabel('x2');
% Plot original data points
for i = 1:4
   if T(i) == 1
      h1=plot(X(i,1), X(i,2), 'go', 'MarkerSize', 10, 'LineWidth', 2); % Green for class 1
   else
      h2=plot(X(i,1), X(i,2), 'rx', 'MarkerSize', 10, 'LineWidth', 2); % Red for class 0
   end
end
% Create a line for the decision boundary: w1*x1 + w2*x2 + b = 0
x1_vals = linspace(-0.5, 1.5, 100);
x2_vals = -(W(1) * x1_vals + b) / W(2);  % Solve for x2
% Plot decision boundary
h3=plot(x1_vals, x2_vals, 'b-', 'LineWidth', 2);
legend([h1 h2 h3], {'Class 1 (Output = 1)', 'Class 0 (Output = 0)', 'Decision Boundary'},
'Location', 'northeast');
axis([-0.5 1.5 -0.5 1.5]);
hold off;
```

Outputs:

Initial Weights: 0.30 -0.10 | Bias: 0.20

Epoch 1

Input: [0 0] | Target: 0 | Output: 1 | Updated W: [0.30 -0.10] | b: 0.10

Input: [0 1] | Target: 0 | Output: 1 | Updated W: [0.30 -0.20] | b: 0.00

Input: [1 0] | Target: 0 | Output: 1 | Updated W: [0.20 -0.20] | b: -0.10

Input: [1 1] | Target: 1 | Output: 0 | Updated W: [0.30 -0.10] | b: 0.00

Epoch 2

Input: [0 0] | Target: 0 | Output: 1 | Updated W: [0.30 -0.10] | b: -0.10

Input: [0 1] | Target: 0 | Output: 0 | Updated W: [0.30 -0.10] | b: -0.10

Input: [1 0] | Target: 0 | Output: 1 | Updated W: [0.20 -0.10] | b: -0.20

Input: [1 1] | Target: 1 | Output: 0 | Updated W: [0.30 0.00] | b: -0.10

Epoch 3

Input: [0 0] | Target: 0 | Output: 0 | Updated W: [0.30 0.00] | b: -0.10

Input: [0 1] | Target: 0 | Output: 0 | Updated W: [0.30 0.00] | b: -0.10

Input: [1 0] | Target: 0 | Output: 1 | Updated W: [0.20 0.00] | b: -0.20

Input: [1 1] | Target: 1 | Output: 0 | Updated W: [0.30 0.10] | b: -0.10

Epoch 4

Input: [0 0] | Target: 0 | Output: 0 | Updated W: [0.30 0.10] | b: -0.10

Input: [0 1] | Target: 0 | Output: 1 | Updated W: [0.30 0.00] | b: -0.20

Input: [1 0] | Target: 0 | Output: 1 | Updated W: [0.20 0.00] | b: -0.30

Input: [1 1] | Target: 1 | Output: 0 | Updated W: [0.30 0.10] | b: -0.20

Epoch 5

Input: [0 0] | Target: 0 | Output: 0 | Updated W: [0.30 0.10] | b: -0.20

Input: [0 1] | Target: 0 | Output: 0 | Updated W: [0.30 0.10] | b: -0.20

Input: [1 0] | Target: 0 | Output: 1 | Updated W: [0.20 0.10] | b: -0.30

Input: [1 1] | Target: 1 | Output: 0 | Updated W: [0.30 0.20] | b: -0.20

Epoch 6

Input: [0 0] | Target: 0 | Output: 0 | Updated W: [0.30 0.20] | b: -0.20

Input: [0 1] | Target: 0 | Output: 0 | Updated W: [0.30 0.20] | b: -0.20

Input: [1 0] | Target: 0 | Output: 1 | Updated W: [0.20 0.20] | b: -0.30

Input: [1 1] | Target: 1 | Output: 1 | Updated W: [0.20 0.20] | b: -0.30

Epoch 7

Input: [0 0] | Target: 0 | Output: 0 | Updated W: [0.20 0.20] | b: -0.30

Input: [0 1] | Target: 0 | Output: 0 | Updated W: [0.20 0.20] | b: -0.30

Input: [1 0] | Target: 0 | Output: 0 | Updated W: [0.20 0.20] | b: -0.30

Input: [1 1] | Target: 1 | Output: 1 | Updated W: [0.20 0.20] | b: -0.30

Training completed successfully.

Final Weights: 0.20 0.20 | Final Bias: -0.30

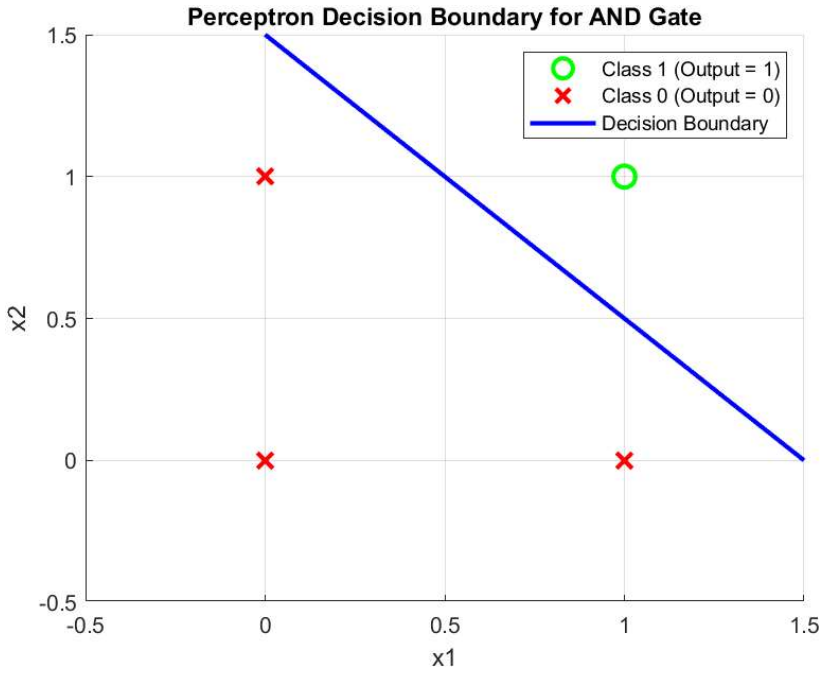
Testing Trained Perceptron:

Input: [0 0] | Output: 0

Input: [0 1] | Output: 0

Input: [1 0] | Output: 0

Input: [1 1] | Output: 1



Results:

The perceptron was successfully implemented and trained to model the behavior of the AND logic gate.

- The initial weights and bias were updated using the **Perceptron Learning Rule**.
- The training converged within a finite number of epochs.
- The final learned weights and bias accurately classified all input patterns of the AND gate.

Ex.No: 03 IMPLEMENTATION OF ARTIFICIAL NEURAL NETWORK Date:

Aim:

To design, train, and evaluate a Feedforward Neural Network using the **bodyfat_dataset** to predict body fat percentage from physiological measurements.

MATLAB Inbuilt Datasets:

Dataset Name	Function to Load	Description
bodyfat_dataset	**[X,T] = bodyfat_dataset;**	**Body fat prediction from body measurements**
cancer_dataset	[X,T] = cancer_dataset;	Breast cancer diagnosis data (benign/malignant)
carbig	load carbig	Car performance data (MPG, horsepower, etc.)
fisheriris	load fisheriris	Iris flower classification (classic dataset)
abalone_dataset	[X,T] = abalone_dataset;	Predict age of abalone from physical measurements
ionosphere	load ionosphere	Radar data for binary classification
simplefit_dataset	[X,T] = simplefit_dataset;	Simple function fitting (X→T)
house_dataset	[X,T] = house_dataset;	Predict house prices from features
maglev_dataset	[X,T] = maglev_dataset;	Magnetic levitation control data
chemical_dataset	[X,T] = chemical_dataset;	Chemical process data for regression
engine_dataset	[X,T] = engine_dataset;	Engine condition monitoring data
ph_dataset	[X,T] = ph_dataset;	pH neutralization process modeling
sine_dataset	[X,T] = sine_dataset;	Sine wave data for time series prediction
mushroom_dataset	[X,T] = mushroom_dataset;	Classification of mushrooms (edible or poisonous)
digittrain_dataset	[X,T] = digittrain_dataset;	Handwritten digit images for classification

Change Training Algorithm

MATLAB uses **trainFcn** to specify the training algorithm.

Example: net.trainFcn = 'traingd'; **% Gradient Descent**

TrainFcn	Description
'trainc'	Perceptron learning rule (default)
'traingd'	Gradient Descent
'traingdm'	Gradient Descent with Momentum
'traingdx'	GD with Momentum and Adaptive LR
'trainlm'	Levenberg-Marquardt (default)
'trainbr'	Bayesian Regularization
'trainrp'	Resilient Backpropagation

Change Performance Function

Set **net.performFcn** to define how performance is calculated during training.

Example: net.performFcn = 'mse'; % **Mean Squared Error**

PerformFcn	Description
'mse'	Mean Squared Error (default)
'mae'	Mean Absolute Error
'sse'	Sum Squared Error
'crossentropy'	For classification tasks

Set Training Parameters

net.trainParam.lr = 0.05; **% Learning Rate**

net.trainParam.epochs = 200; **% Max number of training iterations**

net.trainParam.goal = 0.01**; % Performance goal**

Customize Data Division

By default, data is split into training/validation/testing sets.

Example: net.divideFcn = 'dividerand'; **% Random division (default)**

divideFcn	Description
'dividerand'	Random (default)
'divideblock'	Blocks of sequential data
'divideind'	Manual index-based division
'divideint'	Interleaved blocks
'dividetrain'	Use all data for training only

To manually control the percentages

net.divideParam.trainRatio = 0.7;

net.divideParam.valRatio = 0.15;

net.divideParam.testRatio = 0.15;

MATLAB Transfer Functions (Activation)

Example: net.layers{i}.transferFcn = 'function_name';

Function Name	Full Name	Output Range	Use Case
'logsig'	Logistic Sigmoid	(0, 1)	Binary classification, hidden/output
'tansig'	Hyperbolic Tangent Sigmoid	(−1, 1)	Hidden layers, faster convergence
'purelin'	Linear	(−∞, ∞)	Regression, output layer
'hardlim'	Hard Limit (binary step)	0 or 1	Classic perceptron
'hardlims'	Symmetric Hard Limit	−1 or 1	Simple symmetric binary networks
'satlin'	Saturating Linear (0 to 1)	[0, 1]	Clipping activations
'satlins'	Symmetric Saturating Linear	[−1, 1]	Symmetric clipped outputs
'poslin'	ReLU (Rectified Linear Unit)	[0, ∞)	Deep learning (hidden layers)
'radbas'	Radial Basis	(0, 1)	RBF networks (hidden units)
'softmax'	Softmax	(0, 1), sums to 1	Classification (output layer)
'tribas'	Triangular Basis	(0, 1)	Specialized pattern recognition
'compet'	Competitive	binary vector	Winner-takes-all networks

The Problem: Estimate Body Fat Percentage:

To build a neural network that can estimate the body fat percentage of a person described by thirteen physical attributes:

- Age (years)
- Weight (lbs)
- Height (inches)
- Neck circumference (cm)

- Chest circumference (cm)
- Abdomen circumference (cm)
- Hip circumference (cm)
- Thigh circumference (cm)
- Knee circumference (cm)
- Ankle circumference (cm)
- Biceps (extended) circumference (cm)
- Forearm circumference (cm)
- Wrist circumference (cm)

Algorithm:

1. **Load Dataset**
   - Use the built-in bodyfat_dataset which provides input features (X) and target values (T).
2. **Divide Dataset**
   - Split the dataset into training (70%), validation (15%), and testing (15%) using dividerand.
3. **Create Feedforward Network**
   - Initialize a feedforward neural network using feedforwardnet with one hidden layer of size 10.
4. **Set Data Division**
   - Assign training, validation, and test indices using divideind.
5. **Train the Network**
   - Train the model using train() function.
   - Internally uses backpropagation and gradient descent.
6. **Evaluate Performance**
   - Compute Mean Squared Error (MSE) on entire dataset and specifically on test data.
7. **Visualize Results**
   - Plot:
     - Performance curve
     - Regression plot
     - Training state

- Error histogram

8. **Compare Outputs**

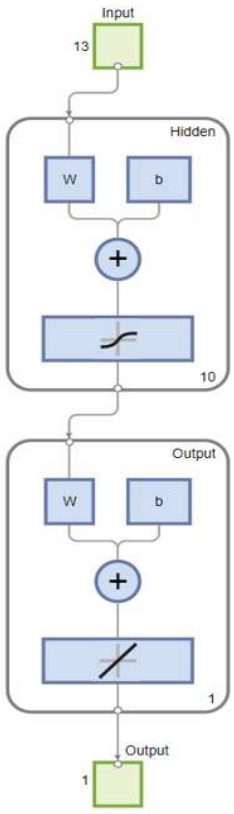
   ➢ Display actual vs predicted output values for test data samples.

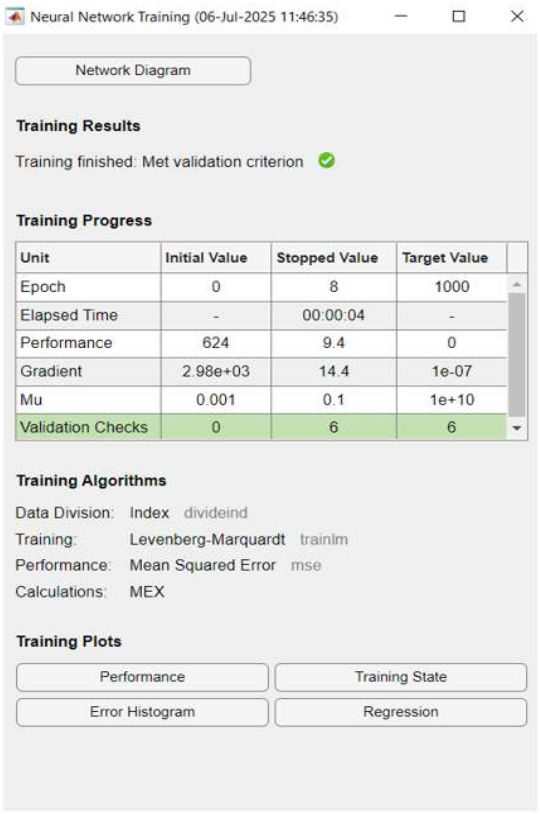
Program:

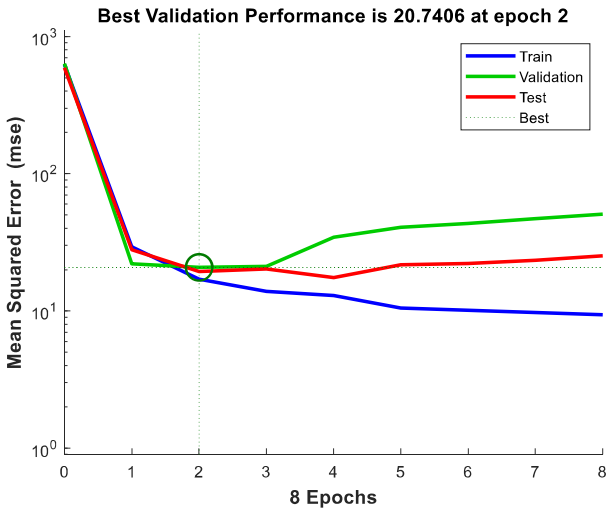
```
clc;
clear;
% Load the body fat dataset
[X, T] = bodyfat_dataset;
% Split into training, validation, and test sets (default is 70/15/15)
[trainInd, valInd, testInd] = dividerand(size(X,2), 0.7, 0.15, 0.15);
% Create a feedforward neural network
hiddenLayerSize = 10;
net = feedforwardnet(hiddenLayerSize);
% Set training, validation, test division manually
net.divideFcn = 'divideind';
net.divideParam.trainInd = trainInd;
net.divideParam.valInd = valInd;
net.divideParam.testInd = testInd;
% Train the network
[net, tr] = train(net, X, T);
% View the network
view(net);
% Test the network
Y = net(X);
perf = perform(net, T, Y);
% Display performance
fprintf('Overall MSE performance: %.4f\n', perf);
% Plot performance
figure, plotperform(tr);
% Plot regression
figure, plotregression(T, Y);
% Plot training state
```

```
figure, plottrainstate(tr);
% Plot error histogram
figure, ploterrhist(T - Y);
% Predict on test samples only
X_test = X(:, tr.testInd);
T_test = T(:, tr.testInd);
Y_test = net(X_test);
test_perf = perform(net, T_test, Y_test);
fprintf('Test MSE: %.4f\n', test_perf);
% Optional: display actual vs predicted
disp('Actual vs Predicted for Test Samples:');
disp(table(T_test', Y_test', 'VariableNames', {'Actual', 'Predicted'}));
```

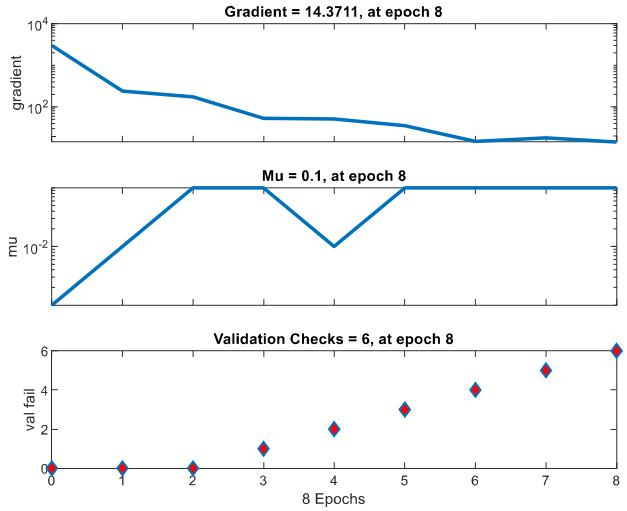
Output:

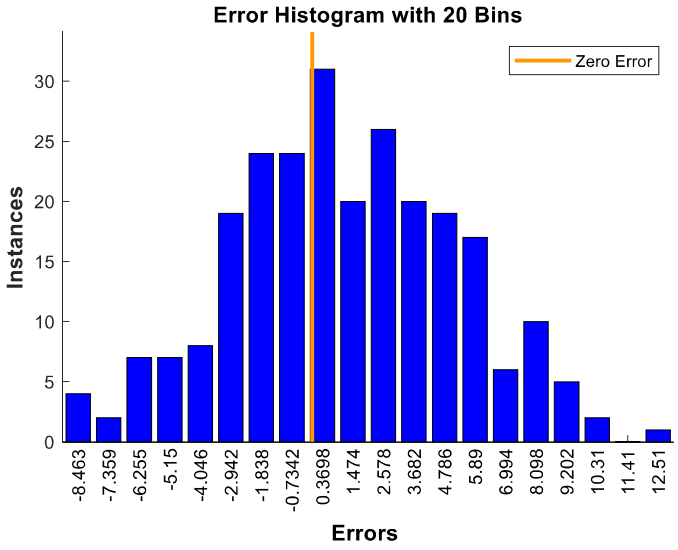


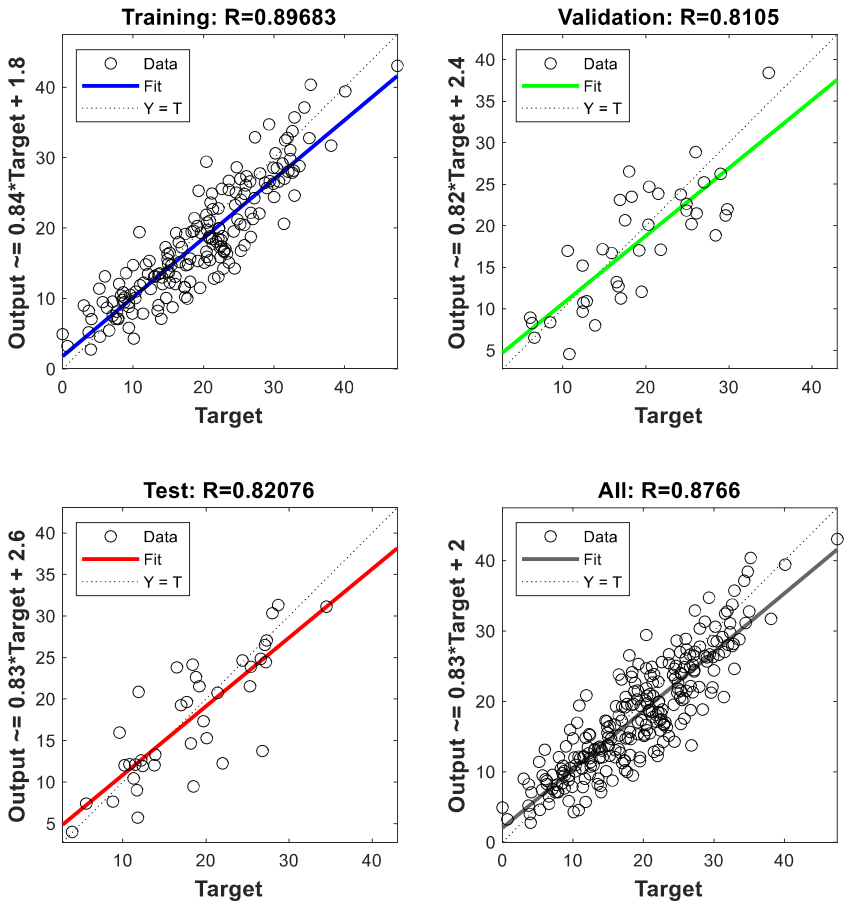




Training State







Overall MSE performance: 17.9338

Test MSE: 19.3455

Result:

The feedforward neural network was successfully trained using the bodyfat_dataset. The architecture consisted of a single hidden layer with 10 neurons. During evaluation, the network achieved an overall Mean Squared Error (MSE) of approximately **17.9338**, and the test set MSE was **19.3455**, indicating a good fit to the data.

Ex.No: 04 **IMPLEMENTATION OF FUZZY SETS** **Date:**

(MEMBERSHIP FUNCTIONS)

Aim:

To implement different fuzzy membership functions (Triangular, Trapezoidal, Gaussian, and Bell-shaped) using MATLAB and to visualize their behavior.

Theory:

Fuzzy Membership Function

A membership function (MF) is a curve that defines how each element in the universe of discourse is mapped to a membership value between 0 and 1.

It represents the degree of truth or degree of belonging of an element to a fuzzy set.

Formally, if X is the universe of discourse, a fuzzy set A is characterized by a membership function:

$$\mu_A : X \to [0,1]$$

Where for each $x \in X$, the function $\mu_A(x)$ gives the membership grade of x in fuzzy set A.

Common Types of Membership Functions:

Triangular Membership Function

$$\mu_A(x) = \begin{cases} 0 & x \le a \text{ or } x \ge c \\ \frac{x-a}{b-a} & a \le x \le b \\ \frac{c-x}{c-b} & b \le x \le c \end{cases}$$

MATLAB: mu = trimf(x,[a b c]);

Trapezoidal Membership Function

$$\mu_A(x) = \begin{cases} 0 & x \le a \text{ or } x \ge d \\ \frac{x-a}{b-a} & a \le x \le b \\ 1 & b \le x \le c \\ \frac{d-x}{d-c} & c \le x \le d \end{cases}$$

MATLAB: mu = trapmf(x,[a b c d]);

Gaussian Membership Function

$$\mu_A(x) = e^{-\frac{(x-c)^2}{2\sigma^2}}$$

Where c is the center and σ is the width.

MATLAB: mu = gaussmf(x,[sigma c]);

Bell-Shaped Membership Function

$$\mu_A(x) = \frac{1}{1 + \left|\frac{x-c}{a}\right|^{2b}}$$

MATLAB: mu = gbellmf(x,[a b c]);

Algorithm:

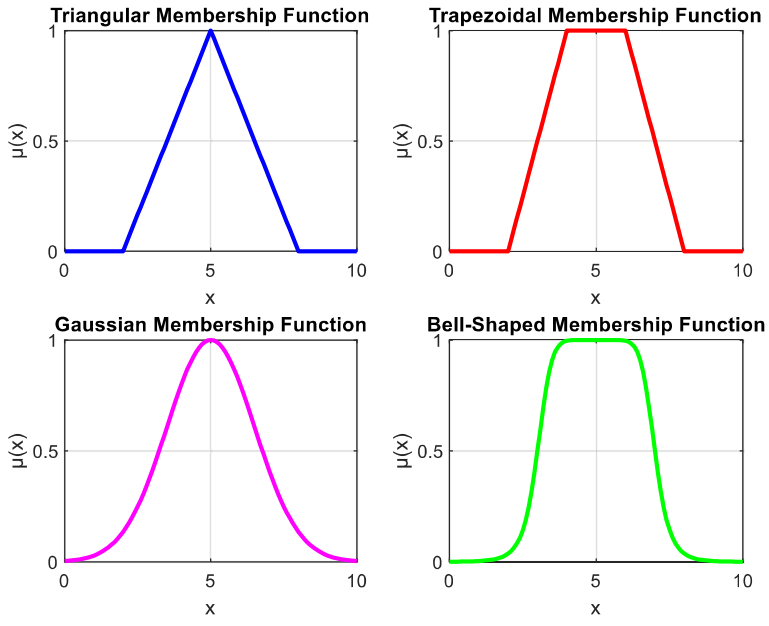
1. Start.
2. Define the universe of discourse x $(e.g., x = 0{:}\,0.1{:}\,10)$.
3. Select the type of membership function (**Triangular, Trapezoidal, Gaussian, or Bell**).
4. Define parameters of the membership function.
5. Use MATLAB functions (**trimf, trapmf, gaussmf, gbellmf**) to generate the MFs.
6. Plot each membership function with proper labels.
7. Stop.

Program:

```
clc;
clear;
x = 0:0.1:10;
% Triangular MF
mu_tri = trimf(x, [2 5 8]);
% Trapezoidal MF
mu_trap = trapmf(x, [2 4 6 8]);
% Gaussian MF
mu_gauss = gaussmf(x, [1.5 5]);
% Bell-Shape MF
mu_bell = gbellmf(x, [2 4 5]);
figure (1);
subplot(2,2,1);
plot(x, mu_tri, 'b', 'LineWidth', 2);
title('Triangular Membership Function');
xlabel('x'); ylabel('μ(x)');
grid on;
subplot(2,2,2);
plot(x, mu_trap, 'r', 'LineWidth', 2);
```

```
title('Trapezoidal Membership Function');
xlabel('x'); ylabel('μ(x)');
grid on;
subplot(2,2,3);
plot(x, mu_gauss, 'm', 'LineWidth', 2);
title('Gaussian Membership Function');
xlabel('x'); ylabel('μ(x)');
grid on;
subplot(2,2,4);
plot(x, mu_bell, 'g', 'LineWidth', 2);
title('Bell-Shaped Membership Function');
xlabel('x'); ylabel('μ(x)');
grid on;
```

Output:



Results:

Different membership functions were successfully implemented in MATLAB. The triangular and trapezoidal membership functions exhibited piecewise linear behavior. The Gaussian membership function produced a smooth bell-shaped curve controlled by its mean and standard deviation. The generalized bell membership function provided a flexible curve with adjustable slope and width. Hence, the behavior of fuzzy sets with different membership functions was visualized successfully.

Ex.No: 05 **IMPLEMENTATION OF FUZZY SETS** **Date:**

(STANDARD, YAGER, SUGENO COMPLEMENTS, UNION & INTERSECTION)

Aim:

To implement fuzzy sets and perform fuzzy set operations such as union, intersection, standard complement, Yager complement, and Sugeno complement using MATLAB.

Theory:

Let A and B be fuzzy sets in universe of discourse $\boldsymbol{X}$, with membership functions $\mu_A(x)$ and $\mu_B(x)$.

Union (OR):

$$\mu_{A\cup B}(x) = \max(\mu_A(x), \mu_B(x))$$

Intersection (AND):

$$\mu_{A\cap B}(x) = \min(\mu_A(x), \mu_B(x))$$

Standard Complement:

$$\mu_{\neg A}(x) = 1 - \mu_A(x)$$

Yager Complement ($\boldsymbol{parameter\ p > 0}$):

$$\mu^{Y}_{\neg A}(x) = \left(1 - \mu_A(x)^p\right)^{\frac{1}{p}}$$

Sugeno Complement ($\boldsymbol{parameter\ a > -1}$):

$$\mu^{S}_{\neg A}(x) = \frac{1 - \mu_A(x)}{1 + a \cdot \mu_A(x)}$$

Algorithm:

1. Start.
2. Define the universe of discourse $(e.g., x = 0{:}\,0.1{:}\,10)$.
3. Define fuzzy sets A and B using Gaussian membership functions.
4. Compute:
   - Standard complement: $1 - \mu(x)$
   - Yager complement with chosen p
   - Sugeno complement with chosen a

- Union using max ()
- Intersection using min ()

5. Plot fuzzy sets and their operations using subplot for clarity.
6. Stop.

Program:

```
clc;
clear;
% Step 1: Define the universe of discourse
x = 0:0.1:10;
% Step 2: Define fuzzy membership functions
% Fuzzy Set A: Gaussian around 3
%mu_A = exp(-((x - 3).^2) / 4);
mu_A = gaussmf(x, [sqrt(2) 3]);
% Fuzzy Set B: Gaussian around 7
%mu_B = exp(-((x - 7).^2) / 4);
mu_B = gaussmf(x, [sqrt(2) 7]);
% Step 3: Fuzzy operations
% Complement of A
mu_complement_A = 1 - mu_A;
% Complement of B
mu_complement_B = 1 - mu_B;
% Yager Complement of A
p = 2;
mu_complement_yager_A = (1 - mu_A.^p).^(1/p);
% Yager Complement of B
p = 2;
mu_complement_yager_B = (1 - mu_B.^p).^(1/p);
% Sugeno Complement of A
a = 0.5;
mu_complement_sugeno_A = (1 - mu_A) ./ (1 + a * mu_A);
% Sugeno Complement of B
a = 0.5
```

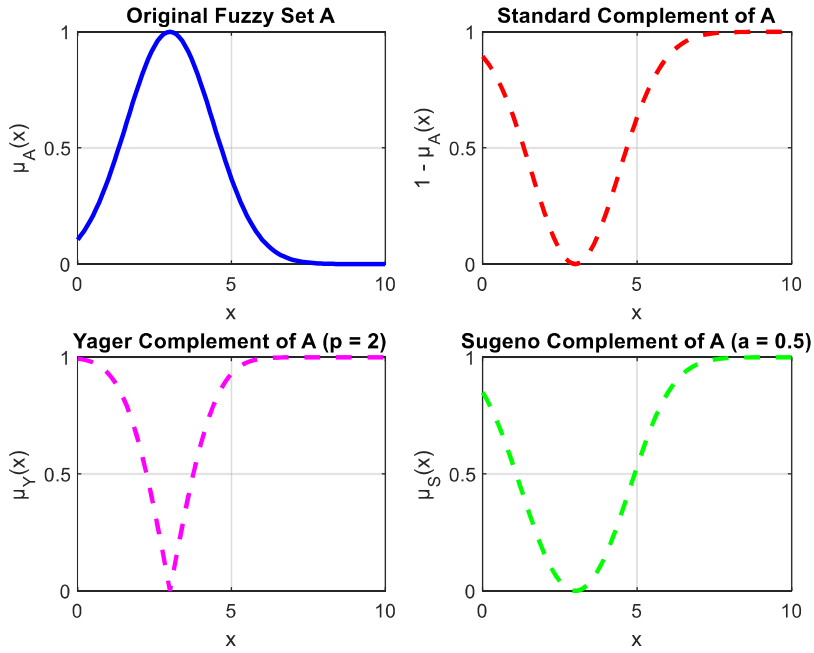
```
mu_complement_sugeno_B = (1 - mu_B) ./ (1 + a * mu_B);
% Union (max)
mu_union = max(mu_A, mu_B);
% Intersection (min)
mu_intersection = min(mu_A, mu_B);
figure (1);
subplot(2,2,1);
plot(x, mu_A, 'b', 'LineWidth', 2);
title('Original Fuzzy Set A');
xlabel('x'); ylabel('μ_A(x)');
grid on;
subplot(2,2,2);
plot(x, mu_complement_A, 'r--', 'LineWidth', 2);
title('Standard Complement of A');
xlabel('x'); ylabel('1 - μ_A(x)');
grid on;
subplot(2,2,3);
plot(x, mu_complement_yager_A, 'm--', 'LineWidth', 2);
title(['Yager Complement of A (p = ' num2str(p) ')']);
xlabel('x'); ylabel('μ_Y(x)');
grid on;
subplot(2,2,4);
plot(x, mu_complement_sugeno_A, 'g--', 'LineWidth', 2);
title(['Sugeno Complement of A (a = ' num2str(a) ')']);
xlabel('x'); ylabel('μ_S(x)');
grid on;
figure (2);
subplot(2,2,1);
plot(x, mu_B, 'b', 'LineWidth', 2);
title('Original Fuzzy Set B');
xlabel('x'); ylabel('μ_B(x)');
grid on;
```

```
subplot(2,2,2);
plot(x, mu_complement_B, 'r--', 'LineWidth', 2);
title('Standard Complement of B');
xlabel('x'); ylabel('1 - μ_B(x)');
grid on;
subplot(2,2,3);
plot(x, mu_complement_yager_B, 'm--', 'LineWidth', 2);
title(['Yager Complement of B (p = ' num2str(p) ')']);
xlabel('x'); ylabel('μ_Y(x)');
grid on;
subplot(2,2,4);
plot(x, mu_complement_sugeno_B, 'g--', 'LineWidth', 2);
title(['Sugeno Complement of B (a = ' num2str(a) ')']);
xlabel('x'); ylabel('μ_S(x)');
grid on;
figure (3);
subplot(2,2,1);
plot(x, mu_A, 'b', 'LineWidth', 2);
title('Fuzzy Set A');
xlabel('x'); ylabel('μ_A(x)');
grid on;
subplot(2,2,2);
plot(x, mu_B, 'g', 'LineWidth', 2);
title('Fuzzy Set B');
xlabel('x'); ylabel('μ_B(x)');
grid on;
subplot(2,2,3);
plot(x, mu_union, '--m', 'LineWidth', 2);
title('Union: A ∪ B');
xlabel('x'); ylabel('μ_{A∪B}(x)');
grid on;
subplot(2,2,4);
plot(x, mu_intersection, '--c', 'LineWidth', 2);
```

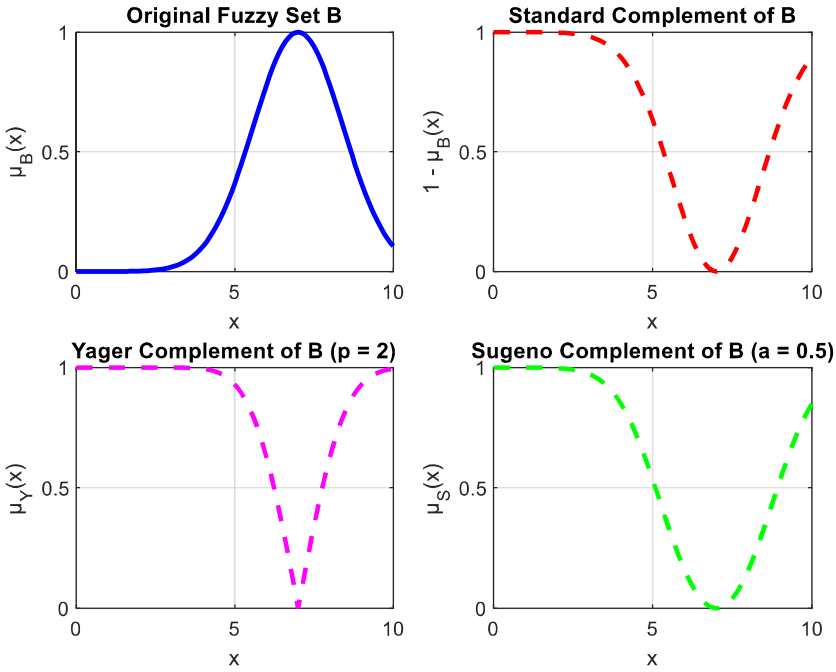
```
title('Intersection: A ∩ B');
xlabel('x'); ylabel('μ_{A∩B}(x)');
grid on;
```

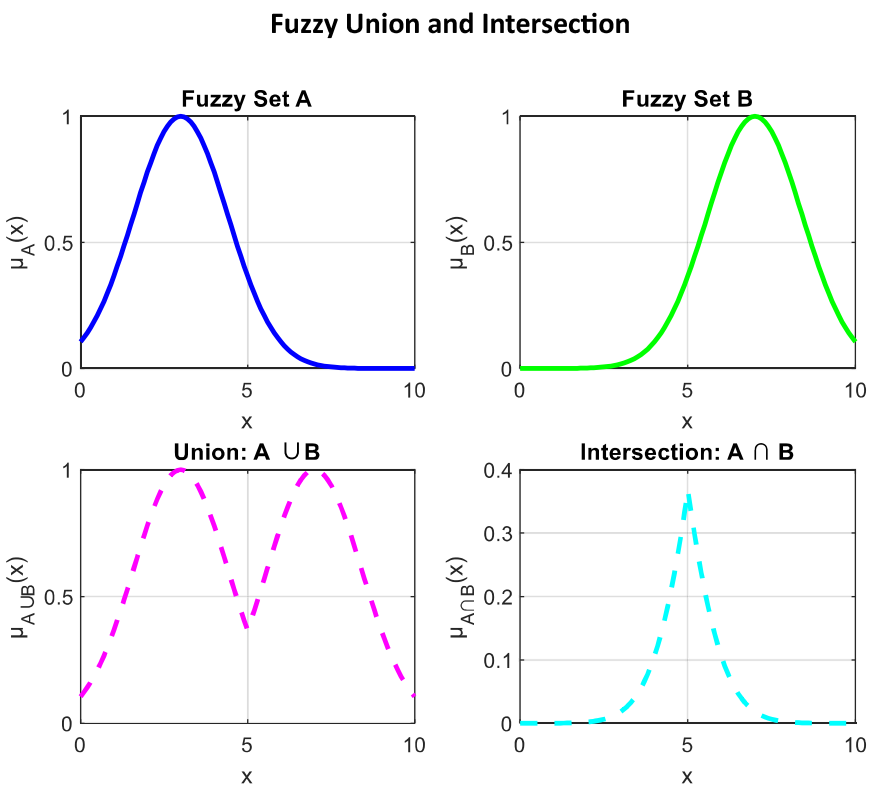
Output:

Fuzzy Set A



Fuzzy Set B





Result:

Fuzzy sets and their operations (union, intersection, standard complement, Yager complement, and Sugeno complement) were successfully implemented and visualized using MATLAB.